Citi Bike Tabular Data Visualization

Requirements & Implementation Specification

Target: Python + Jupyter (VS Code) | Intended reader: developers and Claude Code

How to use this document

This document is the single source of truth for building the code for the assignment *Visualization for Tabular Data - Analyzing New York Citi Bike Data*. It defines the task, the dataset, the project structure, a switchable preprocessing pipeline, and a precise specification for each of the 15 visualizations (a-o). Hand it to Claude Code together with the data file and ask it to implement the project in the order given in Section 11.

Contents

1. Task overview and deliverables
2. Dataset description
3. Tech stack and setup
4. Project structure and organizational pattern
5. Configuration (switches and thresholds)
6. Preprocessing pipeline
7. Shared chart framework
8. Chart specifications (Questions a-o)
9. Report support (per-chart write-up)
10. Coding standards and acceptance criteria
11. Build order for Claude Code

1. Task overview and deliverables

Citi Bike is a bikeshare system in which a bike is unlocked at one docking station and returned at any other. The assignment provides trip records from 2015-2017 and asks for 15 visualizations answering specific questions (a-o). For each visualization the report must also explain:

- (a) the data types shown
- (b) the marks and channels used
- (c) the best practices applied
- (d) the justification for the chart choice and design decisions

Constraints from the assignment

- Allowed tools: Python, Google Sheets, Excel, Flourish, RAWGraphs, Tableau Public. **This project uses Python.**
- Charts must be high quality and high resolution. **Any chart with unreadable labels/text receives zero marks.**
- Final report: concise, **5 pages** (about 3 charts per page). Group submission.

Deliverables of the code

- A clean, documented dataset (Parquet) plus a preprocessing log table.
- 15 PNG charts at 300 DPI in `outputs/charts/`, named by question.
- Summary tables (CSV) behind each chart in `outputs/tables/`, for checking numbers.
- A report-notes file holding the four write-up answers for each chart (Section 9).

2. Dataset description

Source: one Excel file, about **735,500 rows**. Excel holds at most 1,048,576 rows, while the full 2015-2017 system has tens of millions of trips, so this file is a subset.

Important observation: the sample record (station 3212 "Christ Hospital", longitude about -74.05) is a **Jersey City / Hoboken** station. The dataset is most likely the Jersey City Citi Bike data. Implications: station volumes are lower than Manhattan, and the coordinate validation range must suit Jersey City.

Sample record

```
0 | 376 | 10/1/2015 0:16 | 10/1/2015 0:22 | 3212 | Christ Hospital | 40.73478582 | -74.05044364 |
3207 | Oakland Ave | 40.7376037 | -74.0524783 | 24470 | Subscriber | 1960 | 1 | 6
```

Columns

#	Raw column	Clean name	Expected type	Notes
0	(unnamed index)	-	int	Leftover pandas index. Drop it.
1	Trip Duration	trip_duration	int (seconds)	Authoritative duration measure.
2	Start Time	start_time	datetime	Format M/D/YYYY H:MM, minute precision only (no seconds).
3	Stop Time	stop_time	datetime	Same format. Do not recompute duration from times.
4	Start Station ID	start_station_id	int	
5	Start Station Name	start_station_name	string	Strip whitespace.
6-7	Start Station Lat/Long	start_lat, start_lon	float	
8	End Station ID	end_station_id	int	
9	End Station Name	end_station_name	string	
10-11	End Station Lat/Long	end_lat, end_lon	float	

#	Raw column	Clean name	Expected type	Notes
12	Bike ID	bike_id	int	
13	User Type	user_type	category	Subscriber = annual member; Customer = 24h/3-day pass.
14	Birth Year	birth_year	nullable int	Often missing for Customers; may load as float.
15	Gender	gender	int	0 = Unknown, 1 = Male, 2 = Female.
16	(extra column)	trip_minutes_raw	int	Not in the brief. Appears to be duration in minutes (376 s gives 6). Verify across all rows; if confirmed keep as reference, otherwise drop.

Note: the loader must map raw headers to clean names by inspecting the actual header row; if the file has no header row, assign names by position using the table above.

3. Tech stack and setup

Package	Purpose
python >= 3.10	Runtime
pandas, numpy	Data loading, cleaning, aggregation
openpyxl	Reading the .xlsx file (only once)
pyarrow	Fast Parquet save/load of cleaned data
matplotlib, seaborn	All charts
scipy	Correlation (Pearson/Spearman) for Question h
jupyter / ipykernel	Notebooks inside VS Code

Put these in `requirements.txt`. Use a virtual environment (`python -m venv .venv`).

4. Project structure and organizational pattern

Principle: logic in modules, orchestration in notebooks. All functions live in `src/` as importable Python modules. Notebooks stay thin: they import functions, set switches, call them, and display results. This keeps code testable, reusable and easy to change, and avoids long, fragile notebook cells.

```
citibike-viz/
|-- data/
|   |-- raw/                    # original Excel file (never modified)
|   |-- interim/                # raw_converted.parquet (fast copy of Excel)
|   |-- processed/              # citibike_clean.parquet, citibike_with_outliers.parquet
|-- notebooks/
|   |-- 01_data_inspection.ipynb # load, look at shape/dtypes/samples, run validation report
|   |-- 02_preprocessing.ipynb   # run the switchable pipeline, view log, save outputs
|   |-- 03_charts.ipynb          # run selected charts, preview, save PNGs
|-- src/
|   |-- __init__.py
|   |-- config.py               # ALL switches, thresholds, paths, colors (Section 5)
|   |-- io_utils.py             # load_raw_excel, to_parquet, load_data
|   |-- validation.py           # VALIDATION_RULES, validate_data, check_duplicates
|   |-- cleaning.py             # drop_duplicates, drop_invalid, convert_types, standardize_stations
|   |-- outliers.py             # one function per outlier rule (flag or drop)
|   |-- features.py             # add_features
|   |-- pipeline.py             # PipelineLog, run_pipeline (reads config switches)
|   |-- viz_style.py            # set_chart_style, COLORS, save_chart, label helpers
|   |-- charts/
|   |   |-- __init__.py         # CHART_REGISTRY {"qa": (prep_qa, plot_qa), ...}
|   |   |-- qa_duration_hour_month.py
|   |   |-- qb_top_stations_gender.py
|   |   |-- ... one module per question ...
|   |   |-- qo_weekend_vs_weekday.py
|   |-- run_charts.py           # run_all_charts / run_chart
|   |-- report_notes.py         # REPORT_NOTES dict (Section 9)
|-- outputs/
|   |-- charts/                 # qa_*.png ... qo_*.png (300 DPI)
|   |-- tables/                 # qa_*.csv ... summary tables behind each chart
|   |-- logs/                   # preprocessing_log.csv, validation_report.csv
|-- requirements.txt
|-- README.md
```

Notebook conventions

- First cell: `%load_ext autoreload` and `%autoreload 2` so edits in `src/` apply without restarting the kernel; then add the project root to `sys.path`.
- Use Markdown headings per step so the notebook reads like the pipeline.
- Notebooks must run top to bottom with *Restart & Run All*.
- Never write into `data/raw/`.

5. Configuration (switches and thresholds)

Everything a user may want to toggle lives in `src/config.py`. Changing behaviour means editing True/False or a number, never commenting out code.

```
# --- paths ---
RAW_FILE = "data/raw/<file>.xlsx"
INTERIM_FILE = "data/interim/raw_converted.parquet"
CLEAN_FILE = "data/processed/citibike_clean.parquet"
WITH_OUTLIERS_FILE = "data/processed/citibike_with_outliers.parquet"

# --- pipeline switches (True = run step) ---
PIPELINE_STEPS = {
    "drop_duplicates":        True,
    "drop_invalid":           True,
    "standardize_stations":   True,
    "invalid_birth_year":     True,
    "extreme_duration":       True,
    "flag_long_trips":        True,
    "time_errors":            True,
```

```
    "bad_coordinates":        True,
}

# --- per-step action: "drop" removes rows, "flag" only adds a boolean column ---
OUTLIER_ACTION = {
    "invalid_birth_year": "flag",   # keeps rows for non-age charts
    "extreme_duration":   "drop",
    "time_errors":        "drop",
    "bad_coordinates":    "flag",
}

# --- columns that drop_invalid() applies to ---
DROP_INVALID_COLUMNS = ["trip_duration", "start_time", "start_station_id",
                        "end_station_id", "user_type", "gender"]

# --- thresholds ---
MIN_BIRTH_YEAR = 1920
MIN_AGE, MAX_AGE = 16, 80
MIN_DURATION_SEC = 60
EXTREME_DURATION_HOURS = 24
LONG_TRIP_HOURS = 3
LAT_RANGE = (40.60, 40.85)
LON_RANGE = (-74.15, -73.90)
DATE_RANGE = ("2015-01-01", "2017-12-31 23:59")
AGE_BINS   = [16, 25, 35, 45, 55, 65, 81]
AGE_LABELS = ["16-24", "25-34", "35-44", "45-54", "55-64", "65+"]

# --- output versions ---
SAVE_WITH_OUTLIERS_VERSION = True   # also save a dataset with outlier rows kept (flagged only)

# --- chart run settings ---
CHARTS_TO_RUN = ["qa","qb","qc","qd","qe","qf","qg","qh","qi","qj","qk","ql","qm","qn","qo"]
CHART_DATA_VERSION = {"default": "clean", "qn": "with_outliers"}
CHART_PARAMS = {
    "qb": {"top_n": 10}, "qd": {"bin_size": 5}, "qg": {"window": 30},
    "qi": {"top_n": 10}, "qm": {"top_n": 15, "combine_directions": False, "include_round_trips": True},
    "qn": {"split_by_usertype": False, "style": "band"},   # "band" or "box"
    "qo": {"min_daily_trips": 5, "top_n": 15},
    "duration_stat": "median",                             # "median" or "mean", used by qa, qh
}
DPI = 300
```

6. Preprocessing pipeline

Rules for every pipeline function: it takes a DataFrame and returns a new DataFrame (no in-place mutation of the input); it is independent of optional steps; it records what it did in the `PipelineLog` (step, rows before, rows after, rows affected, reason); it has a docstring explaining *what* and *why*. The raw data is reloaded from the interim Parquet at the start of every run, so switching steps on/off always gives a clean re-run.

6.1 Pipeline steps

Step	Function (module)	Switchable	What it does
0	load_raw_excel(), to_parquet() (io_utils)	No	Read Excel once (all sheets; stack if several), drop index column, rename headers to clean names, verify the extra minutes column, save interim Parquet. Later runs load the Parquet.
1	validate_data(df) (validation)	No (check only)	Apply VALIDATION_RULES to every column. Print and return a table: column, rule, failing rows, % of total. Also report missing values per column. Never removes rows. Save to outputs/logs/validation_report.csv.
2	check_duplicates(df), drop_duplicates(df) (validation / cleaning)	Drop: yes	Check prints the count of exact duplicate rows plus examples. Drop removes them, keeping the first.
3	drop_invalid(df, column) (cleaning)	Yes, per column	Remove rows that fail the rule of the given column. Called in a loop over DROP_INVALID_COLUMNS.
4	convert_types(df) (cleaning)	No	Parse datetimes; nullable ints for IDs, birth_year, gender; floats for coordinates; category for user_type; strip text.
5	Outlier functions (outliers)	Yes, each	See 6.3. Each supports action='flag' or 'drop'.
6	standardize_stations(df) (cleaning)	Yes	For each station ID, use its most frequent name so one ID maps to one name.
7	add_features(df) (features)	No	Create derived columns (6.4).
8	PipelineLog.to_frame() (pipeline)	No	Table of all steps with rows before/after and reason; skipped steps shown as 'skipped'. Save to outputs/logs/preprocessing_log.csv (goes into the report).
9	save outputs (io_utils)	No	Save citibike_clean.parquet; if SAVE_WITH_OUTLIERS_VERSION, also save the version where outlier functions only flag.

`run_pipeline(config) -> (df, log)` in `pipeline.py` executes steps 0-9 in order, checking `PIPELINE_STEPS` before each optional step.

6.2 Validation rules (VALIDATION_RULES)

Defined as a dictionary `{column: rule_function}` where each rule returns a boolean Series (True = valid). Used by both `validate_data()` and `drop_invalid()`, so the rules live in one place.

Column	Rule (valid if...)	Typical action on failure
trip_duration	numeric and > 0	drop
start_time	parses as datetime and within DATE_RANGE	drop
stop_time	parses as datetime and >= start_time	drop
start_station_id / end_station_id	numeric, not missing	drop
start_station_name / end_station_name	non-empty string	fill from ID mapping or drop
start_lat/lon, end_lat/lon	numeric, not 0, inside LAT_RANGE / LON_RANGE	flag
bike_id	numeric, not missing	drop
user_type	in {Subscriber, Customer}	drop (or set 'Unknown')
gender	in {0, 1, 2}	drop

Column	Rule (valid if...)	Typical action on failure
birth_year	missing allowed; otherwise numeric and between MIN_BIRTH_YEAR and (trip year - MIN_AGE)	keep row; excluded from age charts
trip_minutes_raw	approximately trip_duration / 60 (difference <= 1)	report only

6.3 Outlier functions (outliers.py)

Each function signature: `func(df, action="flag"|"drop", **thresholds) -> df`. With 'flag' it adds a boolean column; with 'drop' it removes the rows. The docstring states the reason below.

Function	Rule	Flag column	Why it exists
handle_invalid_birth_year	birth_year < MIN_BIRTH_YEAR or age outside MIN_AGE-MAX_AGE	invalid_age (valid_age = not invalid_age)	Ages of 100+ are default or fake entries, not real riders. They distort age distributions.
handle_extreme_duration	trip_duration > EXTREME_DURATION_HOURS	is_extreme_duration	Multi-day trips are undocked bikes or system errors, not rides. One such trip can inflate a mean dramatically.
flag_long_trips	trip_duration > LONG_TRIP_HOURS (always flag only)	is_long_trip	Real but unusual rides (leisure). Kept, so each chart can decide whether to include them.
handle_time_errors	stop_time < start_time or trip_duration <= 0	is_time_error	Physically impossible records.
handle_bad_coordinates	lat/lon missing, 0, or outside configured ranges	is_bad_coord	Invalid locations break maps and station analyses.

Outlier philosophy (for the report): remove only clear *errors*; keep genuine extremes but reduce their influence by using medians, and show them deliberately where useful (Question n box plot).

6.4 Derived columns (add_features)

Column	Definition	Used by
duration_min	trip_duration / 60 (float)	a, h, i, n
year, month, month_name	from start_time; month_name ordered Jan-Dec	a, e, f, j
date	start_time.date()	g, h, l, o
hour	start_time.hour (0-23)	a, l, n
day_of_week	ordered categorical Monday..Sunday	l
is_weekend	day_of_week in {Saturday, Sunday}	o
age	year - birth_year (nullable)	c, d, i
valid_age	age not null and MIN_AGE <= age <= MAX_AGE	c, d, i
age_group	pd.cut(age, AGE_BINS, labels=AGE_LABELS), only where valid_age	c
gender_label	0 -> Unknown, 1 -> Male, 2 -> Female (ordered categorical)	b, d, f, i, k
year_month	start_time.to_period('M')	k
season	Winter (Dec-Feb), Spring (Mar-May), Summer (Jun-Aug), Autumn (Sep-Nov)	h
route	start_station_name + ' -> ' + end_station_name	m
is_round_trip	start_station_id == end_station_id	m

7. Shared chart framework

7.1 viz_style.py

Item	Purpose
set_chart_style()	Set global matplotlib/seaborn style once: font sizes (title 14, axis labels 12, ticks >= 10, never below 9), savefig DPI 300, white background, light grid, despine.
COLORS	Fixed palette used by every chart: user type (Subscriber, Customer); gender (Male, Female, Unknown = grey); season; a sequential colormap for heatmaps. Must be colorblind-safe.
save_chart(fig, name)	Save to outputs/charts/{name}.png at 300 DPI with bbox_inches='tight'.
save_table(df, name)	Save the chart's summary table to outputs/tables/{name}.csv.
format_thousands(ax, axis)	Axis tick formatter, e.g. 12,000 or 12k.
add_bar_labels(ax, fmt)	Value or % labels on bars.

7.2 Prep / plot pattern

Each question is a module in `src/charts/` containing exactly two public functions:

- `prep_qX(df, **params) -> DataFrame`: pandas only; returns a small summary table. No plotting.
- `plot_qX(table, **params) -> Figure`: plotting only; no recalculation.

Benefits: change appearance without recomputing; print the table to check numbers; save the table as evidence.

7.3 Registry and runner (run_charts.py)

```
CHART_REGISTRY = {"qa": (prep_qa, plot_qa, "qa_duration_hour_month"), ...}

def run_chart(key, data_cache):
    version = CHART_DATA_VERSION.get(key, CHART_DATA_VERSION["default"])
    df = data_cache[version]                    # load each version once
    params = CHART_PARAMS.get(key, {})
    table = prep(df, **params); save_table(table, name)
    fig = plot(table, **params); save_chart(fig, name)
    return table, fig

def run_all_charts():
    for key in CHARTS_TO_RUN: run_chart(key, data_cache)
```

8. Chart specifications (Questions a-o)

Each spec lists the columns, the prep logic, the plot (library and core call), parameters and output name. All charts use `COLORS`, clear titles, axis labels with units, and readable fonts.

8.a Question a

Question	How does the average trip duration change throughout the day across different months?
Columns	month, hour, duration_min
prep_qa()	Group by month x hour; aggregate duration_min with duration_stat (median default). Pivot: rows = months Jan-Dec (all years combined), columns = hours 0-23.
plot_qa()	seaborn.heatmap(table, cmap=sequential, cbar_kws={'label': 'Median duration (min)'}). X = hour, Y = month.
Params	duration_stat
Output	qa_duration_hour_month

8.b Question b

Question	Which stations have the highest traffic (trip frequency), and how does this vary by gender?
Columns	start_station_name, gender_label
prep_qb()	Count trips per start station; keep top_n. Filter to those stations; count by station x gender_label; pivot gender to columns; sort by total. Also compute % female per station.
plot_qb()	pandas DataFrame.plot.barh(stacked=True) (matplotlib). Segments Male / Female / Unknown (grey); largest station at top; optional % female labels.
Params	top_n=10
Output	qb_top_stations_gender
Notes	Traffic = trips starting at the station (state this in the report).

8.c Question c

Question	What is the distribution of user types across different age groups?
Columns	age_group, user_type, valid_age
prep_qc()	Filter valid_age. Count by age_group x user_type; normalize to % within each age group. Also return the count of Customers with missing birth year (caveat for report).
plot_qc()	100% stacked bar: DataFrame.plot.bar(stacked=True); X = age group, segments = user type; % labels.
Params	-
Output	qc_usertype_by_agegroup

8.d Question d

Question	What is the age and gender distribution of Citi Bike users?
Columns	age, gender_label, valid_age
prep_qd()	Filter valid_age and gender in {Male, Female}. Bin age into bin_size-year bins (new column age_bin). Count by age_bin x gender; make Male counts negative.
plot_qd()	Population pyramid: two matplotlib ax.barh() calls (Male left, Female right), Y = age bins. X tick labels shown as absolute values.
Params	bin_size=5
Output	qd_age_gender_pyramid

8.e Question e

Question	What percentage of trips are made by each user type over the three years?
Columns	year, user_type
prep_qe()	Count by year x user_type; % within each year.
plot_qe()	100% stacked bar per year (2015, 2016, 2017); ax.bar_label() for % labels.
Params	-
Output	qe_usertype_share_by_year

8.f Question f

Question	What proportion of rides were taken by each gender in each year?
Columns	year, gender_label
prep_qf()	Count by year x gender_label; % within each year.
plot_qf()	100% stacked bar per year with % labels; Unknown in grey.
Params	-
Output	qf_gender_share_by_year

8.g Question g

Question	How has the number of daily rides changed over the three years?
Columns	date
prep_qg()	Count trips per date; reindex to the full date range filling 0; add rolling mean column rolling_{window}.
plot_qg()	matplotlib ax.plot() twice: daily (thin, light, alpha) and rolling mean (thick, dark). Date-formatted X axis (quarters/months); optional annotations for notable dips.
Params	window=30
Output	qg_daily_rides_trend
Notes	Counts are sample counts (state in report).

8.h Question h

Question	Is there a relationship between the total number of daily trips and the average trip duration?
Columns	date, duration_min, season
prep_qh()	Per date: daily_trips (count) and avg_duration (duration_stat); attach season. Compute Pearson and Spearman r (scipy.stats); return table and r values.
plot_qh()	seaborn.scatterplot(x=daily_trips, y=avg_duration, hue=season) + seaborn.regplot(scatter=False) for trend; r shown as text annotation.
Params	duration_stat
Output	qh_trips_vs_duration

8.i Question i

Question	How do factors like trip duration, user type, gender and age affect start station selection?
Columns	start_station_name, duration_min, user_type, gender_label, age, valid_age
prep_qi()	For top_n start stations compute: median duration, % Subscriber, % Female (of known gender), median age (valid ages). Return actual table and a z-score normalized copy (per column).
plot_qi()	seaborn.heatmap(normalized, annot=actual_formatted, fmt='', cmap=diverging, center=0). Rows = stations, columns = factors. Cells show real values (e.g. '12.4 min', '81%').
Params	top_n=10
Output	qi_station_factor_profile
Notes	Alternative (optional): 4 small-multiple bar charts, one per factor.

8.j Question j

Question	How do trips by user type vary over the months in each year?
Columns	year, month, user_type
prep_qj()	Count by year x month x user_type (long format).
plot_qj()	seaborn.relplot(kind='line', x=month, y=count, hue=user_type, col=year, col_wrap=3, facet_kws={'sharey': True}).
Params	-
Output	qj_usertype_monthly_by_year

8.k Question k

Question	How has the proportion of trips each gender takes changed over time?
Columns	year_month, gender_label
prep_qk()	Count by year_month x gender_label; % within each month; pivot gender to columns (36 rows).
plot_qk()	matplotlib ax.stackplot() (100% stacked area); X = month timeline; same gender colors as b and f.
Params	-
Output	qk_gender_share_over_time

8.l Question l

Question	What times of day experience the highest trip activity across different days of the week?
Columns	day_of_week, hour, date
prep_ql()	Count by day_of_week x hour; divide by number of distinct dates of each weekday (average trips per hour). Pivot: rows Monday..Sunday, columns 0-23.
plot_ql()	seaborn.heatmap(table, cmap=sequential, cbar label 'Avg trips per hour').
Params	-
Output	ql_weekday_hour_heatmap

8.m Question m

Question	Which station pairs are most frequently used?
Columns	start_station_name, end_station_name, route, is_round_trip
prep_qm()	If combine_directions: sort the two names alphabetically before joining. If not include_round_trips: exclude is_round_trip. Count by route; keep top_n.
plot_qm()	Sorted horizontal bar: ax.barh(); route names on Y, count labels on bars.
Params	top_n=15, combine_directions=False, include_round_trips=True
Output	qm_top_station_pairs

8.n Question n

Question	Are there certain times of day when trips tend to be longer?
Columns	hour, duration_min, optionally user_type
prep_qn()	Per hour (and user_type if split): median, 25th and 75th percentiles. For style='box', return a random sample (e.g. 300k rows) of hour + duration_min instead.
plot_qn()	style='band': ax.plot(median) + ax.fill_between(q25, q75). style='box': seaborn.boxplot(x=hour, y=duration_min) with log-scaled or capped Y.
Params	split_by_usertype=False, style='band'
Output	qn_duration_by_hour
Notes	Default data version 'with_outliers' so outliers can be shown deliberately.

8.o Question o

Question	Do certain stations experience higher traffic on weekends than on weekdays?
Columns	start_station_name, is_weekend, date
prep_qo()	Count distinct weekend and weekday dates. Count trips per station x is_weekend; divide by the matching day count, giving avg_daily_weekday and avg_daily_weekend. weekend_ratio = weekend / weekday. Filter stations with total avg daily trips < min_daily_trips. Keep top_n by ratio.
plot_qo()	Dumbbell chart: ax.hlines() connecting the two values + two ax.scatter() calls (weekday / weekend), stations sorted by ratio.
Params	min_daily_trips=5, top_n=15
Output	qo_weekend_vs_weekday
Notes	Normalizing per day is essential (5 weekdays vs 2 weekend days).

9. Report support (per-chart write-up)

The report must answer four questions for each chart. Store them in `src/report_notes.py` as a dictionary so the team can fill and reuse them, and so `03_charts.ipynb` can display each note under its chart.

```
REPORT_NOTES = {
  "qa": {
    "data_types":     "Ordinal (month, hour); Quantitative (median duration)",
    "marks_channels": "Mark: area (cell). Channels: x-position (hour), y-position (month),
                       colour luminance (duration)",
    "best_practices": "Sequential colormap, labelled colour bar with units, median for skewed data, ...",
    "justification":  "Heatmap shows two ordinal dimensions against one quantity compactly; ...",
  },
  ...
}
```

Claude Code should pre-fill a draft for all 15 entries based on the specs in Section 8; the team then refines them after seeing the actual charts. Also export the preprocessing log as a compact table for the report.

10. Coding standards and acceptance criteria

Standards

- Every function has a docstring (purpose, inputs, outputs, and for outliers the reason).
- Functions never mutate their input; they return a new DataFrame.
- No hard-coded thresholds, paths or colors inside functions; read from config.py.
- Use categorical ordering for month, day_of_week, age_group, gender_label so charts sort correctly.
- Use median for duration summaries unless duration_stat says otherwise.
- Print short, readable summaries (counts and %) rather than dumping large frames.

Acceptance criteria

- `01_data_inspection.ipynb` prints shape, dtypes, a sample, the validation report and the duplicate count, and confirms the meaning of the extra minutes column.
- `02_preprocessing.ipynb` runs end to end; toggling any switch in config.py and re-running changes the log accordingly with no other edits.
- Preprocessing log shows every step, rows before/after and reason; skipped steps are marked 'skipped'.
- `03_charts.ipynb` produces all 15 PNGs at 300 DPI and 15 CSV tables; changing CHARTS_TO_RUN runs only those charts.
- Every chart has a title, axis labels with units, a legend where needed, consistent colors, and no text smaller than 9 pt at the final size.
- Percentages within each group sum to 100 (+/- 0.1) where charts are 100% stacked.

11. Build order for Claude Code

#	Task	Done when
1	Create folder structure, requirements.txt, README.md, empty modules.	Tree matches Section 4.
2	Write config.py exactly as Section 5 (adapt RAW_FILE).	Imports without error.
3	io_utils.py: load_raw_excel (header mapping, drop index col), to_parquet, load_data(version).	Interim Parquet created; load is fast.
4	validation.py: VALIDATION_RULES, validate_data, check_duplicates. Build 01_data_inspection.ipynb.	Validation table printed and saved.
5	cleaning.py, outliers.py, features.py, pipeline.py (PipelineLog, run_pipeline). Build 02_preprocessing.ipynb.	Clean Parquet(s) and log saved; switches work.
6	viz_style.py and run_charts.py with registry.	Style applies; save helpers work.

#	Task	Done when
7	Chart modules qa-qo following Section 8, one at a time, checking each output.	15 PNG + 15 CSV.
8	report_notes.py with draft notes; 03_charts.ipynb shows chart + table head + notes per question.	Notebook runs top to bottom.

Suggested prompt for Claude Code: *"Read this requirements PDF. Implement the project in VS Code following Section 11 step by step. After each step, run the code and show me the result before continuing. The data file is at data/raw/<file>.xlsx."*